

Why UEFI Security Matters

UEFI firmware runs before the operating system loads, creating a critical attack surface that traditional security solutions cannot protect against. Vulnerabilities at this level can lead to persistent malware that survives OS reinstalls and bypasses security measures.

-  Runs at **Ring -2** (highest privilege)
-  Controls **Secure Boot** process
-  Direct access to **all hardware**
-  Malware **persists** across OS reinstalls
-  Invisible to **OS-level** security tools

Understanding UEFI

What is UEFI?

Unified Extensible Firmware Interface - modern replacement for legacy BIOS

Firmware interface between operating system and platform firmware

Provides standardized environment for booting an OS and running pre-boot applications

UEFI vs. BIOS

UEFI Boot Process

1 SEC (Security)

Initial platform startup, CPU reset, chipset initialization

2

PEI (Pre-EFI Initialization)

Memory initialization, temporary RAM setup

3

DXE (Driver Execution Environment)

Permanent memory setup, driver dispatching

4

BDS (Boot Device Selection)

Boot policy enforcement, boot option selection

5

RT (Runtime)

Handoff to OS, runtime services available

Security Features

-  Secure Boot
-  Measured Boot
-  TPM Integration
-  Firmware Authentication
-  Platform Key Hierarchy
-  SMM Protections

CPU Protection Rings and Privilege Levels

What Are Protection Rings?

Hierarchical privilege levels in CPU architecture that enforce **access control** and protect critical system resources

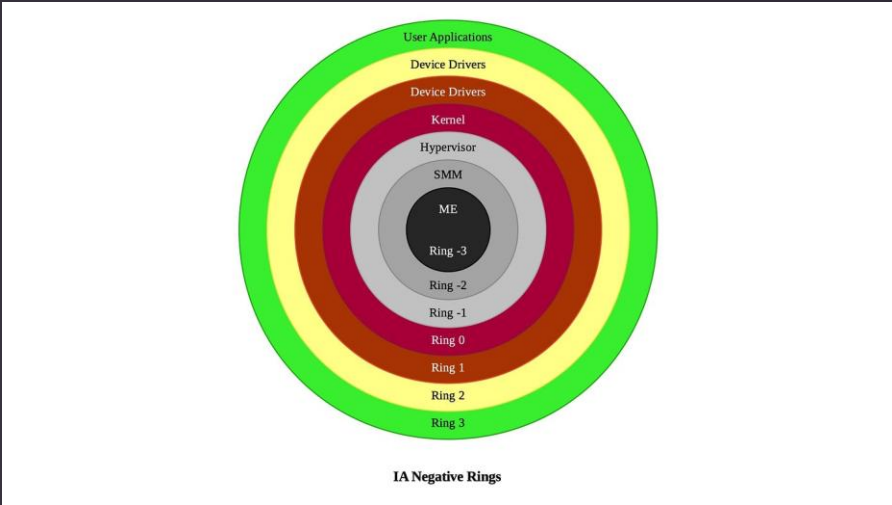
Lower numbers = **higher privilege** and more system access

Each ring has restrictions on memory access and CPU instructions

Ring Privileges & Functions

Ring	Privilege	Function
Ring -2	Highest	System Management Mode (SMM)
Ring -1	Very High	Hypervisor / VMM
Ring 0	High	OS Kernel / Drivers
Ring 1 & 2	Medium	Device Drivers (rarely used)
Ring 3	Lowest	User Applications

Privilege Hierarchy



Security Implications

- > Ring -2 (SMM) operates above OS visibility, making exploits extremely difficult to detect
- > Compromise at **higher privilege rings** can bypass all lower-level security controls
- > **SMM code** has unrestricted access to all physical memory, hardware, and CPU state

Rootkits and Bootkits

Types of Malicious Kits



Rootkits

Malicious software designed to **hide its existence** while maintaining privileged access



Bootkits

Malware that infects the **boot process** to load before the operating system



Firmware Kits

Malware embedded in **UEFI firmware** that survives OS reinstallation



Kernel Kits

Malware operating at **Ring 0** with full system access

Feature	Rootkits	Bootkits
Activation Time	After OS loads	Before OS loads
Persistence	File system	Boot sector
Detection Difficulty	High	Very High
Removal	OS reinstall	Firmware reflash



UEFI Rootkits & Bootkits

UEFI rootkits represent the **most dangerous** class of firmware malware, operating at the lowest level of system execution



Invisible to OS - operates below operating system visibility



Survives reformatting - persists across OS reinstalls



Bypasses Secure Boot - can disable security mechanisms



Full system control - complete access to hardware and memory



Persistence Mechanisms



UEFI Variable Corruption - Modifying NVRAM variables to maintain persistence



DXE Driver Infection - Infecting UEFI drivers that load during boot



SPI Flash Modification - Directly modifying firmware on flash chip



SMM Code Injection - Exploiting SMM vulnerabilities for execution

System Management Mode (SMM) Security

What is SMM?

System Management Mode - A special-purpose execution mode in x86 CPUs that operates at the **highest privilege level** (Ring -2)

Designed to handle system-wide functions like power management and hardware control

Completely **transparent to the OS** and all other software

Why SMM is a Security Concern

-  **Invisible to OS** - No visibility or control from operating system
-  **Unrestricted Access** - Complete control over all physical memory and hardware
-  **Bypasses Protections** - Circumvents OS security, hypervisors, and Secure Boot
-  **Complex Code** - Difficult to audit and often contains vulnerabilities

SMM Purposes

-  **Power Management** - Thermal control, fan speed, sleep states
-  **Hardware Control** - Emulating missing hardware features
-  **System Management** - OEM-specific system functions
-  **Error Handling** - Critical system error recovery
-  **Security Functions** - TPM operations, secure boot

Why Arbitrary Code Execution in SMM is Extremely Dangerous

-  **Persistence** - Malware survives OS reinstallation and hard drive wipes
-  **Full System Control** - Complete access to encryption keys, passwords, and all data
-  **Undetectable** - Traditional security tools cannot see or remove SMM malware
-  **Network Compromise** - Can establish covert communication channels

SMM Vulnerability Impact

SMM exploits can lead to **permanent system compromise** that can only be resolved by reflashing the firmware with a clean image

Technical Details

</> SMM Handler Architecture



SMI Handler Registration

SMM handlers register to respond to specific System Management Interrupts (SMIs)



SMRAM Protection

Special memory region accessible only in SMM, protected by CPU features



Communication Buffer

Data exchange mechanism between SMM and non-SMM code

```
EFI_STATUS EFI_API SmmGetVariable (  
    IN CHAR16 *VariableName,  
    IN EFI_GUID *VendorGuid,  
    OUT UINT32 *Attributes,  
    IN OUT UINTN *DataSize,  
    OUT VOID *Data  
) // Vulnerable function
```



Vulnerability Exploitation Flow

1

Trigger SMI

Attacker sends specially crafted request via **USB/Thunderbolt** device to trigger vulnerable SMM handler

2

Overflow VariableName Buffer

Excessively long **VariableName** parameter overwrites stack buffer and adjacent memory

3

Control SMM Execution

Overwritten return address redirects execution to attacker-controlled code in **SMRAM**

4

Escalate Privileges

Malicious code executes with **Ring -2** privileges, bypassing all OS security

5

Establish Persistence

Modify UEFI firmware to maintain persistence across reboots and OS reinstalls

Vulnerability Overview

🔑	CVE ID:	CVE-2025-27718
⚠️	Severity:	CVSS 8.8 HIGH
💻	Affected:	Dell + Gigabyte BIOS < 184.108.40.206
🛡️	Vector:	Physical access required
🕒	Published:	August 2025

🔧 Stack Buffer Overflow Vulnerability

- 📏 Insufficient Length Validation
 - 📝 `VariableName` length not properly validated before copying to stack buffer
- 📄 Unsafe Memory Operations
 - 📝 Using `CopyMem()` without bounds checking allows overflow
- ➡️ Control Flow Hijacking
 - 📝 Overflow overwrites return address, redirecting execution to attacker's payload

⚙️ Technical Trigger

- ▶️ `Stack buffer overflow` in `SmmGetVariable()` handler
- ▶️ Exploitable via malicious USB/Thunderbolt device
- ▶️ Allows arbitrary code execution in `System Management Mode (SMM)`
- ▶️ Vulnerability exists in variable name length validation
- ▶️ Attacker can overwrite SMI handler execution flow

Attack Scenario

1

Initial Access

Attacker boots target system with **crafted USB/Thunderbolt device** containing malicious payload

2

Trigger Vulnerability

Malicious SMI (System Management Interrupt) triggers **stack buffer overflow** in SmmGetVariable() handler

3

Code Execution

Overflow allows **arbitrary code execution** in System Management Mode (SMM)

4

Persistence

Malicious code modifies UEFI firmware to install **persistent rootkit** that survives OS reinstallation

⚠ Impact



Bypasses Secure Boot and other OS security mechanisms



Rootkit remains **invisible** to OS-level security tools



Attacker gains **full system control** with highest privileges



Malware **persists** across OS reinstalls and hard drive wipes



Potential for **data theft** and long-term surveillance

Detection & Mitigation

Detection Tools



Binarily IQ

Commercial firmware scanner that detects **SMM vulnerabilities** and malicious code patterns



CHIPSEC

Open-source framework for analyzing **platform security** including firmware, hardware, and configuration



fwupd

Open-source daemon for managing firmware updates on Linux, includes **security checks**



UEFI Scanner

Analyzes UEFI firmware images for **known vulnerabilities** and suspicious components

Mitigation Strategies



Update BIOS

Install Dell BIOS version **≥ 220.127.116.11** or later to patch the vulnerability



Enable UEFI Secure Boot

Prevents unauthorized firmware from loading during boot process



Enable SMM Protection

Activate **SMM_BWP** and **BIOSWE** protection in BIOS settings



Restrict Physical Access

Limit USB/Thunderbolt port access to prevent **physical attacks**



Firmware Attestation

Implement **remote attestation** to verify firmware integrity

Broader Implications

Industry Impact



Cross-Vendor Vulnerabilities

Similar flaws discovered in **Lenovo, HP, and Gigabyte** firmware



Supply Chain Risks

Firmware compromised during **manufacturing process** or distribution



Security Erosion

Undermines trust in **Secure Boot** and other firmware protections



Regulatory Scrutiny

Increased **government oversight** of firmware security practices

Future Trends



Pre-OS Security Focus

Enhanced **firmware attestation** and measurement technologies



Hardware-Based Protections

Intel CET and **AMD SEV** providing stronger isolation



SMM Hardening

Stricter **SMM code signing** and privilege separation



Standardized Security

Industry-wide **security frameworks** for firmware development

Q&A & Resources

Questions?



We're here to help

Feel free to ask any questions about UEFI security, the vulnerability, or mitigation strategies

Additional Resources



Video: "This Goes Deeper Than Your Operating System"

Deep dive into motherboard exploits and UEFI security by Low Level
youtube.com/watch?v=BTjj1ILCwRs



Advisory: Binarly BRLY-2025-009

Technical analysis of the SMM memory corruption vulnerability
binarly.io/advisories/brly-2025-009



Microsoft: System Management Mode Deep Dive

How SMM isolation hardens the platform against attacks
microsoft.com/security/blog/2020/11/12/smm-deep-dive



Tools: CHIPSEC Framework

Open-source platform security assessment framework
github.com/chipsec/chipsec